

Chapter #8

Q. Techniques to improve Classification Accuracy?

Ensemble Learning

⇒ An ensemble for classification is a composite model, made up of a combination of classifiers.

Q. Explain the basic characteristics of ensemble learning methodologies?

⇒ One of the primary goals of data mining is to predict an "unknown" value.

⇒ A new sample from observed samples.

⇒ Such a prediction is achieved

by two sequential phases:

1. Training Phase

2. Testing Phase

1. Training Phase:

Objective:

⇒ Produce a predictive model from training samples using a supervised learning algorithm.

Process:

⇒ Utilizes available supervised learning algorithms to generate a predictive model based on

Observed samples.

Testing Phase:

Objective:

⇒ Evaluate the generated predictive model using test samples that were not used in the training phase.

2. Methods:

⇒ Assess the performance of the predictive model using independent test samples to validate its accuracy and reliability.

No-Free Lunch Theorem

Principles:

⇒ There is no single algorithm that is universally superior in all applications.

Explanation:

⇒ Each algorithm operates based on certain assumptions, which may or may not hold true in different scenarios.

Implications:

⇒ No algorithm consistently outperforms others, leading to the need for multiple approaches.

Ensemble Learning:

Goal:

⇒ Improve the accuracy of predictive models by combining results from various models generated using training samples.

Approach:

⇒ Aggregate predictions from diverse predictive models to reduce error rates.

Motivation:

⇒ Leveraging the wisdom of multiple models to enhance decision-making and decrease misclassification.

Theoretical Validations

Hypothesis:

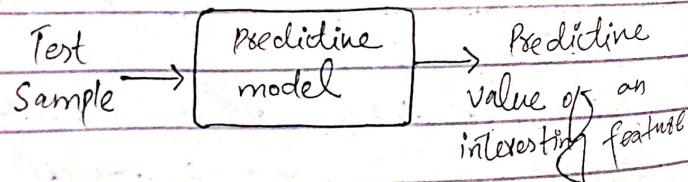
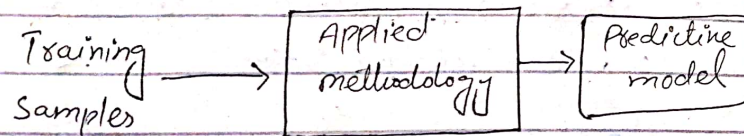
⇒ Ensemble learning enhances accuracy by aggregating predictions from multiple, independent and diverse models.

Examples

- ⇒ Combining ensemble performance to individual performance in decision making scenarios, such as:
- ⇒ Estimating the number of jelly beans in a jar?

Theoretical Proof

- ⇒ Harsanyi and colleagues demonstrated that if N classifiers make independent errors with an error probability $e < 0.5$, the error of an ensemble decreases monotonically as a function of N .
- ⇒ However, performance deteriorates rapidly with dependent classifiers.



Ensemble Compositions

Components:

- ⇒ An ensemble comprises several predictive models (e.g., Predictive model₁, Predictive model₂, ..., Predictive model_n) and a combining rule.

- ⇒ Ensemble learning is also referred to as a combination of multiple classifiers, classifier fusion, mixture of experts, or consensus aggregation.

Requirements for Ensemble performance:

Independence:

- Predictive models should be independent, with uncorrelated errors.

Accuracy:

- Each model should have an accuracy rate > 0.5 for effective ensemble performance.

Ensemble Error rate Calculation:

Calculation:

⇒ The error rate of ensemble is significantly lower than that of a single classifier when each model's error rate is < 0.5 .

Example:

⇒ For an ensemble of 15 classifiers with an error rate of 0.3 each, the ensemble's error rate is calculated as the sum of combinations where at least eight models misclassify a sample.

Effect of Predictive model Number and Error Rate:

Trend:

⇒ Increasing the number of predictive models in an ensemble reduces the ensemble's error rate, especially when individual model error rates are < 0.5 .

Illustration:

⇒ Error-rate curves show a

decrease in ensemble error rate with larger numbers of predictive models.

Q. How to generate base learners and how to combine the output from base learners?

Generating Diverse Base Learners:

⇒ Diverse and independent learners can be generated by:

- Using different learning algorithms for different learning models such as SVM, decision trees and neural networks.
- Use different hyper-parameters in the same algorithm to tune different models.
- Use different input representations, such as using different subsets of input features in a data set.
- Using different training subsets of input data to generate different models usually using the same learning methodology.

Stacked Generalization

Methodology

⇒ Combines models of different types by introducing a meta-learner.

Procedure

⇒ The learning procedure is as follows:-

1. Split the training set into two disjoint sets.
2. Train several base learners on the first part.
3. Test the base learners on the second part.
4. Using the predictions from (3) as the inputs, and the correct responses as the outputs, train a higher level learner.

Methodological Variations

⇒ Some methodologies are based on manual or automatic feature selection that can be used for

generating diverse classifiers using different feature sets.

1. Random Subspace Method:

⇒ It is based on the theory of Stochastic discrimination.

⇒ Learning machines are trained on randomly chosen subspaces of the original input space and the outputs of the models are then combined.

Purpose

⇒ Reduce overfitting and improve model generalization in machine learning.

Approach

Feature Subsetting

⇒ Randomly select subsets of features from the full feature set.

Model Training

⇒ Train multiple models using these randomly selected feature subsets.

Diversity
⇒ Each model specializes in different aspects of the data due to the varied feature subsets.

Ensemble prediction:

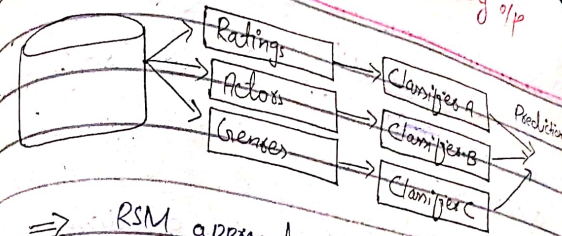
⇒ Combine predictions from all models to make the final prediction.
⇒ This integration of diverse models helps in making more accurate predictions.

Benefit

⇒ RSM works well for large feature sets with redundant features.
⇒ Reduces overfitting by introducing diversity among base learners.
⇒ Helps in handling varying importance levels of individual features.

overfitting :-

Training time less o/p
but Test time wrong o/p



⇒ RSM approach in ensemble classifiers for movie classification.

2nd - Decision tree for C & R.

2. Random Forest:

bagging algorithm - voting based
- extension of decision tree
- multiple decision trees created and make prediction

⇒ Random forest is an ensemble learning method in data mining and machine learning that operates by constructing multiple decision trees during training and outputting the class that is the mode of the classes of the individual trees.

⇒ Consisting of collection of decision tree classifiers. Remove overfitting problem of decision trees.

⇒ The name of "forest" refers to this ensemble of decision trees.

Steps: ⇒ more accurate for classification

1. Data sampling:

⇒ Randomly select subsets of the training data with replacement. This is

⇒ Divide data set randomly into small parts and process

called bootstrapping.

randomly choose data

⇒ Each subset is used to train a decision tree.

Feature selections

⇒ At each node of the decision tree, only random subset of features is considered for splitting.

⇒ The randomness helps to decorrelate the trees and reduce overfitting.

Random Attribute Selection

⇒ During the construction of each decision tree, a random subset of attributes is considered as candidates for splitting at each node.

⇒ This subset is typically much smaller than the total number of available attributes.

Building Decision Trees

⇒ For each subset of data, grow a decision tree by recursively splitting the data based on the selected features.

make full sized decision tree
- sometimes might be bigger than other but there is no predetermined depth

⇒ The splits are chosen to maximize the information gain or minimize impurity.

⇒ Decision trees in random forests are grown to maximum size and are not pruned.

⇒ The CART methodology is often used to grow the trees.

Random Forest Variants

⇒ Two variants of random forests are:

- o Forest - RI - which uses random input selection.

- o Forest - RC - which creates new attributes as linear combinations of existing attributes.

Comparing Random Forests to AdaBoost

⇒ Random forests are comparable in accuracy to AdaBoost but are more robust to cross and outliers.

They also converge to generalization error as the number of trees in the forest increases.

Generalization and Overfitting in Random Forests

⇒ Random forests are less prone to overfitting due to the ensemble of decision trees and the randomness in attribute selection.

⇒ They converge to a stable performance with increasing numbers of trees.

Factors Affecting Accuracy in Random Forests

⇒ The accuracy of a random forest depends on the strength of individual classifiers and the measure of dependence between them.

⇒ Maintaining the strength of individual classifiers while

reducing their correlation is ideal.

Efficiency of Random Forests on Large Databases

⇒ Random forests are efficient on large databases because they consider fewer attributes for each split compared to other methods like bagging or boosting.

⇒ This efficiency makes them suitable for processing large datasets.

Internal Estimates of Variable Importance in Random Forests

⇒ Random forests provide internal estimates of variable importance, which can be useful for feature selection and understanding the factors that contribute most to the predictions.

Benefits:

- High Accuracy
- Robustness
- Feature importance
- Efficiency

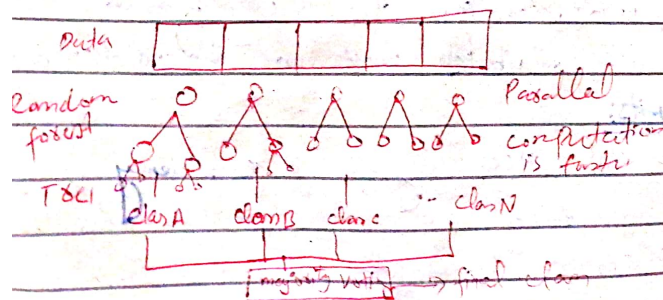
Applications:

⇒ Random forest is commonly used in various applications such as:

- Classification tasks like spam detection, image classification, and medical diagnosis.
- Regression tasks like predicting house prices, stock prices and demand forecasting.

Conclusions:

⇒ Random forest is a versatile and powerful algorithm in data mining, known for its simplicity, scalability, and effectiveness in handling complex datasets.



2. Introduce AdaBoost algorithm and its advantages. And also tell what is boosting?

Boosting:

⇒ Boosting belongs to ensemble learning family.

⇒ It is sequential ensemble learning method.

⇒ Originally designed for classification, but can also be extended for the regression tasks.

⇒ The algorithm starts by creating a "weak" classifiers, slightly better than random sampling/guessing.

⇒ Each training sample is given a weight, usually starting equally.

⇒ It works iteratively training weak models on different subsets of data and give more weight to instances that were misclassified in previous iterations.

⇒ During each learning step:

- Increase weights of the samples that are correctly learned by the weak learners.

⇒ Decrease weights of the samples that are correctly learned by the weak learners.

⇒ Each subsequent model focuses on correcting errors made by previous ones.

⇒ The final prediction is weighted sum of individual model predictions.

Algorithm

Step 1: Each training tuple in boosting is assigned weights.

Step 2: A sequence of k classifiers is iteratively trained.

Step 3: After learning a classifier m_i , weights are updated to focus more on tuples misclassified by m_i .

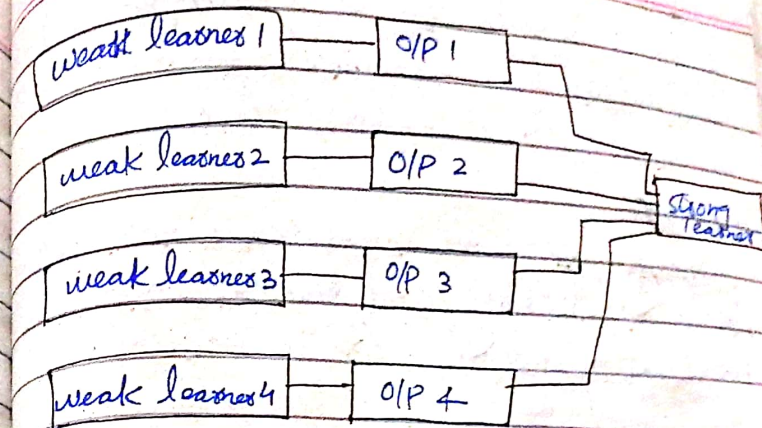
Step 4: The final boosted classifier, M^* , combines the votes for all classifiers, with each classifier's vote weighted based on its accuracy.

Boosting Algorithms:

- o AdaBoost

- o Gradient Boosting

- o XGBoost



Key Concept for Adaboost Algorithm

Weak learners:

⇒ A weak learner is a model whose accuracy is very low.

⇒ It is a simple model that performs slightly better than random guessing on a given task.

⇒ However, they still contribute to the ensemble's performance when combined with other weak learners through techniques like boosting or bagging.

⇒ Weak learners are often simple and computationally inexpensive, such as decision stumps or shallow neural networks.

⇒ They are used in ensemble methods to collectively improve the overall predictive performance of the model.

Decision stumps (Order is important)

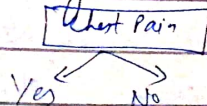
⇒ A decision stump is a simple decision tree model that consists of only one decision node and two leaf nodes. Some stumps say more in the classification.

⇒ Whose maximum depth is one. it makes predictions by evaluating the value of single feature and applying a simple rule.

⇒ Decision stumps are shallow and have limited predictive power compared to full decision trees, but they are computationally efficient and easy to interpret.

- Not making accurate classification.

- They are weak learners.



AdaBoost Algorithm

→ 1996

Creates forest of (stumps) from scratch
- Leaves has a single node and two lines

⇒ AdaBoost stands for Adaptive boosting. it is a boosting strategy that is applied in ML as a part of an ensemble method.

⇒ it belongs to supervised learning algorithm.

⇒ AdaBoost is a boosting set of rules that was added with the aid of Yoav Freund and Robert Schapire in 1996.

⇒ it combines weak learners to create a strong, high-quality learner.

⇒ Key Concepts

⇒ Instead of sampling like bagging, AdaBoost reweights samples.

⇒ it repeatedly uses the same training set and may add weak learners until a target training error is reached.

Main steps

⇒ Train weak classifiers with the training data.

- ⇒ It repeatedly uses the same training set.
- ⇒ Classify each sample and adjust weights based on misclassifications.
- ⇒ Combine weak classifiers into a strong classifier.

How does AdaBoost work?

- ⇒ The AdaBoost algorithm is iterative. The algorithm starts by training a weak classifier on the training data.
- ⇒ The weak learner/classifier is then weighted according to how well it performs.
- ⇒ The algorithm then trains a second weak classifier using the weighted data and then weights.
- ⇒ This process is repeated for a given number of iterations or until the error rate is below a threshold.
- ⇒ The final classifier is a weighted average of all the weak classifiers. The lower the error rate, the higher the weight.

Steps of AdaBoost Algorithm

Initialization:

Initialize the weight of each tuple in D to $1/d$.

Training phase/iterations:

- For $i = 1$ to K (for each round):
- Sample D with replacement according to the tuple weights to obtain D_i .
- Use training set D_i to derive a model, M_i .
- Compute errors (M_i), the error rate of M_i .
- If errors (M_i) > 0.5 then go back to (b).

Weight update:

- For each tuple in D_i that was correctly classified:
 - Multiply the weight of the tuple by $\frac{\text{errors}(M_i)}{1 - \text{errors}(M_i)}$ (update weights)
- Normalize the weight of each tuple.

Ensemble Classification:

- ⇒ Initialize weight of each class to 0.
- o for each classifier $i = 1$ to k :
 - o $w_i = \frac{\log(1 - \text{error}(m_i))}{\text{error}(m_i)}$ (weight of the classifier's vote).
- o $c = M_i(x)$ (get prediction for x from M_i)
- o Add w_i to weight for class c .
- ⇒ Return the class with the largest weight.

Advantages:

- o simple to understand and implement.
- o Versatile and can be used for a variety of tasks such as regression, classification, and outlier detection.
- o improved accuracy
- o feature selection based
- o Resistance to overfitting
- o Requires minimal parameter tuning
- o Effective for complex classification problems.

Disadvantages:

- ⇒ Sensitivity to noisy data
- ⇒ Computationally intensive
- ⇒ Less prone to overfitting
- ⇒ Selecting the proper learning and tuning hyperparameters may be difficult.

Practical Application:

- o Face detection
- o Speech Recognition
- o Anomaly Detection

Example

Age	Income	Buy Product
30	50000	0
35	60000	1
25	40000	0
40	70000	1
45	80000	1
20	30000	0
55	90000	1
50	85000	1
28	45000	0
32	55000	1

Two features are age and income.

Age: Age of the person in years.

Income: Annual income of the person (in dollars).

Buy product: Binary label indicating whether person bought the product or not (0).

⇒ This dataset can be used as i/p to train an AdaBoost classifier to predict whether a person will buy a product based on their age and income features.

Step 1: Initialization

- Each example is initially assigned a weight of $1/n$.

Step 2: Training weak learners

- Train a decision stump on the data set.

- Decision stump chooses the feature and threshold that best separates data. Assume the decision stump chooses the feature "Age" and threshold "35".

Step 3: Evaluate weak learners

- Suppose the decision stump misclassifies 2 examples.
- The error of the weak learner is $2/10 = 0.2$.

The weight of weak learner is calculated as:
$$\alpha = \frac{1}{2} \log \left(\frac{1-0.2}{0.2} \right) \approx 0.693.$$

Step 4: update data weights:

- Increase the weights of misclassified examples
- And decrease the weights of correctly classified examples.

Step 5: Repeat:

- Repeat steps 2-4 for a specified number of iterations.

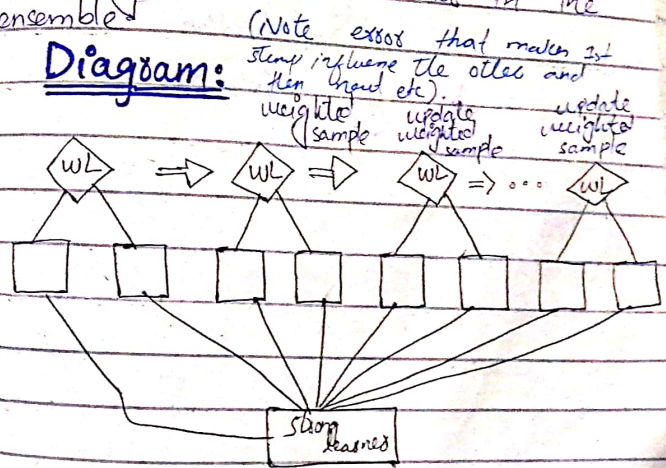
Step 6: Combine weak learners

- Combine the weak learners into a strong classifier.

Step 7: final classification:

- Classify new data points by taking a weighted vote of the predictions made by each weak learner in the ensemble.

Diagram:



Conclusions

- To summarize AdaBoost algorithm is:
- Sequential
- Computational scalability
- Used to exploit/use the dependency between models.
- Stagewise Additive multimodeling using multiclass exponential loss function.
- Decision stumps
- Can handle missing values and outliers.
- Can handle mixed predictions as well (quantitative and qualitative).
- Performance of stumps:
$$\text{New weights} = \text{old weights} \times e^{\pm \text{Performance}}$$
 - + for misclassification
 - for right classification.

Bagging

Bagging stands for bootstrap aggregation. It was the first effective method of ensemble learning and is one of the simplest methods. It was originally designed for classification and is usually applied to decision tree models, and adaptable for any type of model both regression and classification.

⇒ Applicability

Methodology

- ⇒ Utilizes multiple versions of a training set created via bootstrap sampling (sampling with replacement).
- ⇒ Each sample set trains different model.
- ⇒ Outputs are combined through averaging (for regression) or voting (for classification).

Training dataset formation:

⇒ Sample taken with replacement from an initial set according to a sampling distribution.

⇒ Sampling distribution determines the likelihood of selection.

Effectiveness:

⇒ Effective with unstable nonlinear models. Small changes in training data lead to significantly different classifiers and accuracy changes.

Purpose:

⇒ Decreases error by reducing variance in results of unstable learners.

Algorithm:

1. D , a set of training tuples
 k , the number of models in ensemble.

a classification learning scheme (decision tree algorithm, naïve Bayesian).

2. a composite model M .

Method:

for $i = 1$ to k // create k models:

1. create bootstrap sample D_i by sampling D with replacement
2. use D_i and the learning scheme to derive a model m_i ;
3. endfor

Stepwise explanation:

Initialization:

• for each iteration i from 1 to k .
Bootstrap sample creation:

• Create a bootstrap sample D_i by SNR.

Model training:

• Utilize D_i and the chosen classification learning scheme to derive a model m_i .

• Each bootstrap sample D_i represents a different subset of the training data.

• The chosen learning scheme trains a classifier on D_i .

Ensemble operations

⇒ Repeat steps 2 and 3 for k iterations to create a set of k models $\{M_1, M_2, \dots, M_k\}$.

Classifications

⇒ To classify a new tuple x :

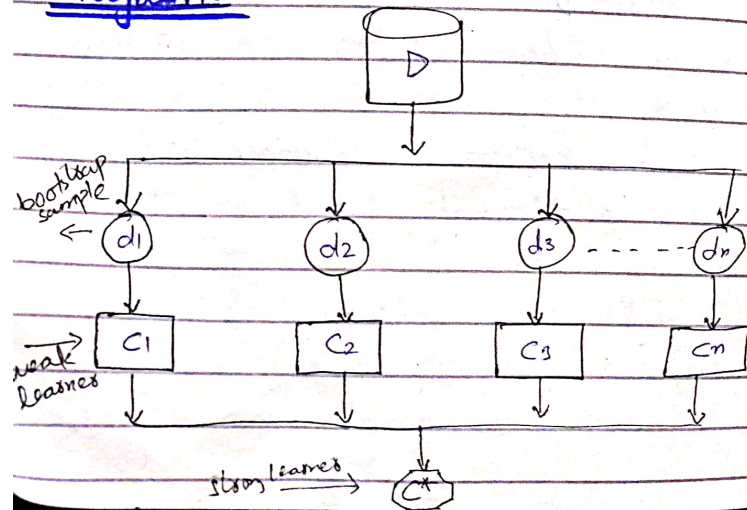
⇒ Let each of the k models classify x and return the majority vote.

⇒ Each model independently predicts the class of x .

⇒ The final prediction is determined by the majority vote among all models.

⇒ Equal weighting ensures that each model has an equal say in the final decision.

Diagram:



Example:

Random forest Algorithm

(Solved already on previous Pg)

Benefits:

- Reduce variance and overfitting by combining diverse models.
- Enhance accuracy.

Cons:

- Increase Complexity
- Reduced interpretability
- Sensitive to Noise
- Scalability

Bagging VS Boosting

Bagging

uses random subsets of the training data with replacement.

Purpose:

Helps reduce overfitting.

Best for:

Best for unstable models with high variance.

weights

All models get equal weight.

Aim:

Aims to reduce variance.

Combines similar type of model predictions.

Models are built independently.

Boosting

uses subsets that focus on misclassified data from previous models.

Helps reduce bias.

Best for stable models with high bias.

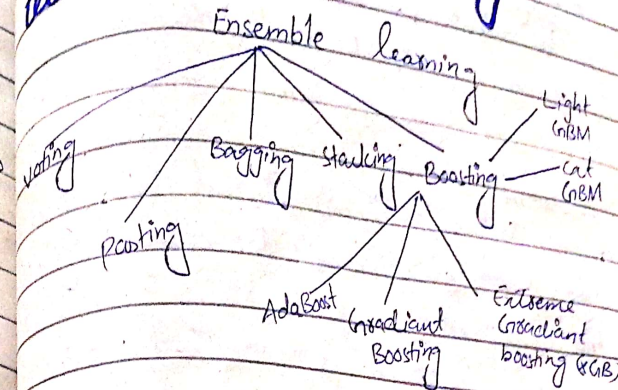
models are weighted based on performance.

Aims to reduce bias.

Combines different types of model predictions.

New models depend on the performance of earlier models.

Q. List Ensemble learning techniques.



Combination Scheme for multiple learners:

=> include:

1. Global approach (Learners fusion):

=> In this approach, every learner in the system produce an output for each i/p pattern. These outputs are then combined to make the final decision. These are several ways to combine the o/p.

Voting:

Each classifier votes for a certain class, and the class with the most votes wins.

y : Final decision
 d_j : output of j classifier
 w_j : weight assigned

Averaging

- o The outputs of all classifiers are averaged, and the final decision is based on this average.

Stacking

- o A new model is trained to learn how best to combine the outputs of all the classifiers.
- o This approach ensures that every classifier contributes to the final decision for each i/p.

2. Local approach (Learner selection)

- o it involves selecting one or a few classifiers that are best suited for a particular i/p pattern. The selection is based on how closely the classifiers match the i/p.

Selection function:

- o This function decides which classifiers to use based on their performance or other criteria related to the i/p system.
- o Not all classifiers contribute to every decision.

3. Multistage Combination:

- o In multistage combination, approach, classifiers are arranged in a sequence, and each stage deals with the instances that the previous stages found challenging.
- o This process works as follows:
 → The first classifier tries to classify the i/p. If the first classifier is unsure or makes an error, the i/p is passed to the next classifier in the sequence.

- o This continues until the i/p is classified correctly or all classifiers have been used.
- o This serial approach ensures that each stage learns from the mistakes of the previous stages progressively refining the decision.

4. Rank-level fusion method:

- o Some classifiers provide not just a single class label but a ranked list of class labels or class probabilities.
- o Rank-level fusion methods combine these ranked lists to make the final decision.